Building UI with React

useEffect + fetch
Technical Debt!

Managing Server State with TanStack Query

Think Beyond the UI!

Client State

UI Data Only

Modal — Theme — Tabs

Server State

Remote Data

Users — Orders — Notifications

The Challenges of Server State

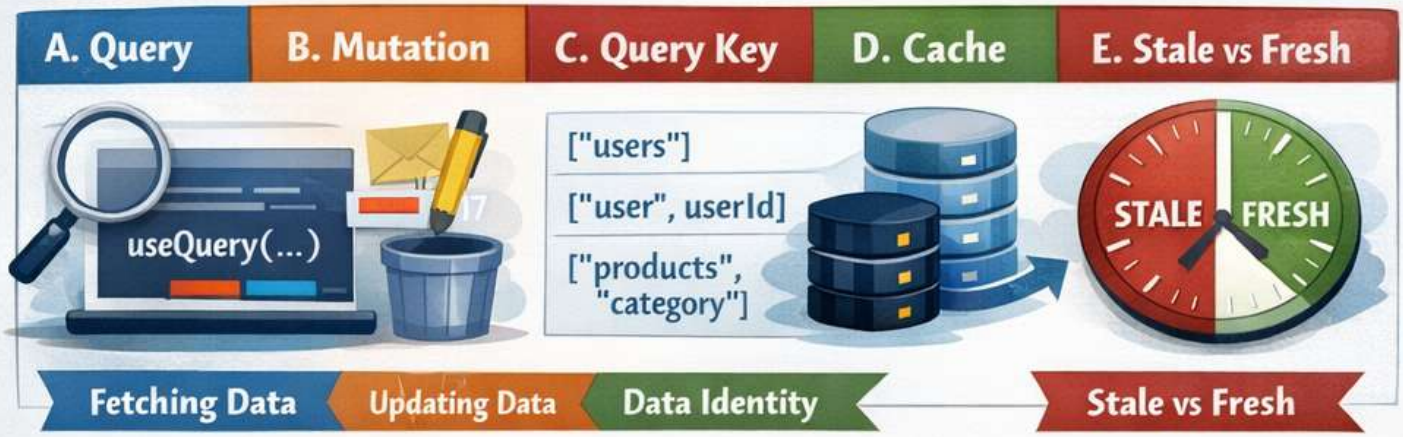
Fetching — Caching — Syncing — Stale Data — Retries — Background Updates

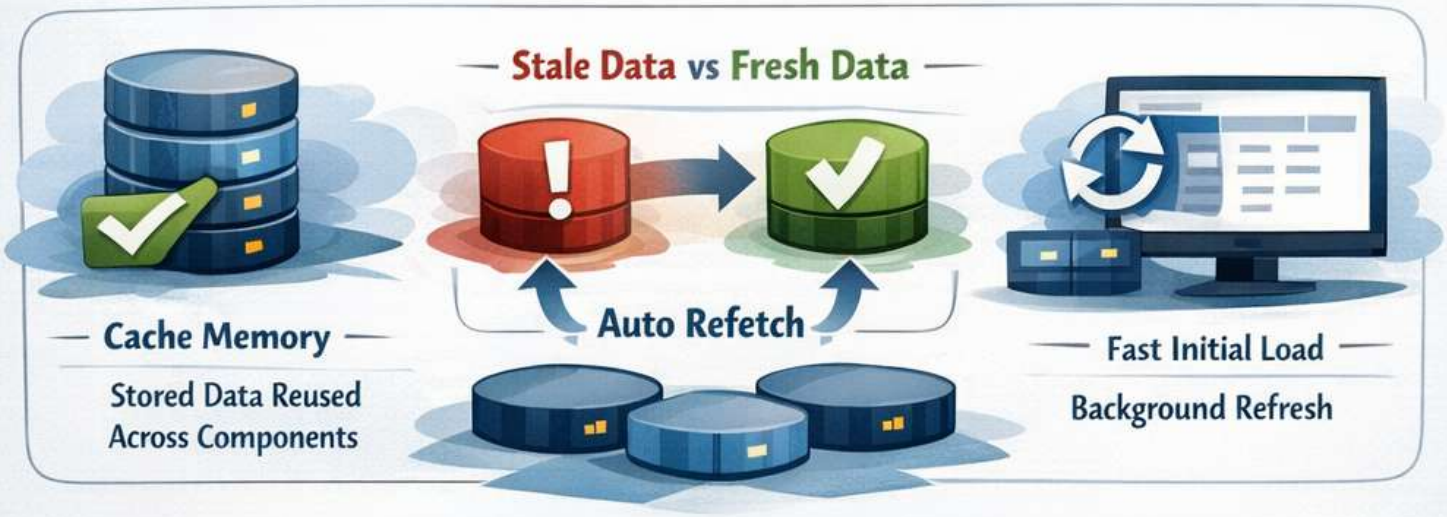
React Query to the Rescue!

- ✅ Smart Caching
- ✅ Auto Refetching
- ✅ Error Handling
- ✅ Data Sync
- ✅ Mutation & Pagination

The Core Mental Model

— 5 Key Concepts of React Query —





Query Keys — Learn This Properly

5 Key Concepts of React Query

Bad Query Keys

- ["data"]
- ["stuff"]

Too Vague

Good Query Keys

- ["users"]
- ["user", userId']
- ["products", category"]
- ["orders", {status "pending"']

Clear & Specific

Why Query Keys Matter

- Cache Identity
- Deduplication
- Invalidation
- Refetch Targeting

Example:

```
useQuery({
    queryKey: ["user", 5],
    queryFn: () -> fetchUserByDId(5),
});
```

Cache for User 5 — Cache for User 6

Separate Caches, Separate Resources!

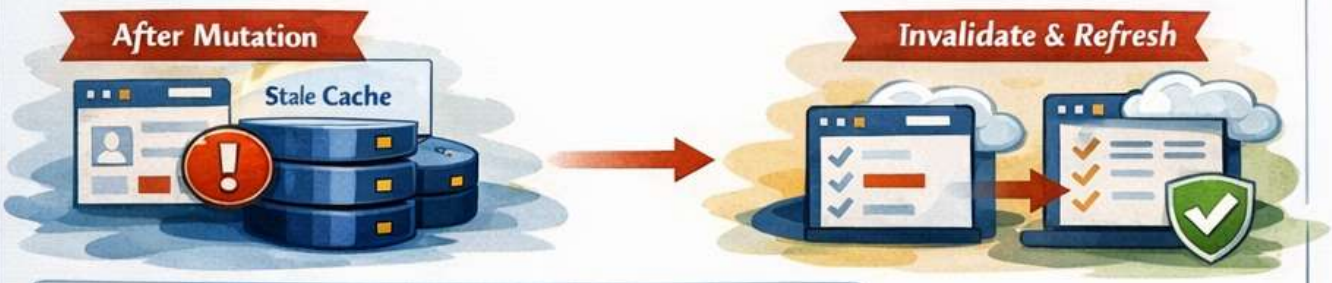
Mutations — Writing to the Server

Turning Actions into Reality!

Basic useMutation

```
const mutation = useMutation{
      mutationFn: async (newUser) =>{
      // API Call...
}};
```

The Power of Invalidation




```
queryClient. invalidateQueries({"users");
});
```

Cache Updated!

Conditional Fetching with *enabled*

Control When to Fetch Data

✗ Bad Query Keys ✗

Only Fetch When Ready!

✓ Good Query Keys ✓

```
["users"]
["user", userId'],
["products", category]
["orders", [stars "piending"]
```

Now Safe to Fetch!

```
const { data } = useQuery((
        queryKey: ["user", userId],
        queryFn: { ) –> fetchUserById(userId!)),
        enabled: !!userId,
});
```

Common Use Cases

Modal Open | Item Selected | Valid Form | Auth Token Available

Be Careful! Guard Your Queries

Don't Trigger Invalid Requests!

Separate Caches, Separate Resources!

Recommended Folder Structure

Organizing Code for Scalability

Folder Pattern

```
src/
  services/          → API Logic Here
    api.ts
    users.ts
  hooks/             → Custom Hooks Here
    queries/
      useUsers.ts
      useUser.ts
  mutations/
    useCreateUser.ts
    useDeleteUser.ts
```

Clean Component Example

```
const mutation = useMutation(
    mutationFnc async (newUser) -> {
    // API Call...
  }});
```

useUsers Hook

```
▸ useQuery(["users"])
▸ fetchUsers()
```

- ✔ Fetching & Caching
- ✔ Query Logic
- ✔ Reusable Code

```
queryClient, invaiidateQueries("users");
```

Why It's Better

Separation of Concerns

Messy Code

Maintainable Code

Scales with Growth!